

Java Server Pages (JSP)



BÀI 11: LẬP TRÌNH JSP

- ❑ JSTL Core Tags
- ❑ JSP Standard Actions
- ❑ Scope trong JSP và Servlet
- ❑ Bài toán tổng hợp



Các thẻ JSTL cơ bản (JSTL Core Tags)

Thẻ	Mô tả
<c:out>	Để viết một cái gì đó trong trang JSP , có thể sử dụng EL cũng có thẻ này
<c:import>	Giống với <jsp:include> hoặc chỉ thị include (include directive)
<c:redirect>	Chuyển hướng (redirect) yêu cầu tới một nguồn dữ liệu khác.
<c:set>	Set đặt giá trị biến cho bởi phạm vi.
<c:remove>	Loại bỏ biến ra khỏi phạm vi đã cho.
<c:catch>	Bắt ngoại lệ và gói vào một đối tượng.
<c:if>	Điều kiện logic đơn giản, sử dụng với EL và bạn có thể sử dụng nó để xử lý các trường hợp ngoại lệ từ <c:catch>



Các thẻ JSTL cơ bản (JSTL Core Tags)

Thẻ	Mô tả
<c:choose>	Tag có điều kiện đơn giản mà thiết lập một bối cảnh cho các trường hợp loại trừ lẫn nhau có điều kiện, đánh dấu bằng <c:when> và <c:otherwise>
<c:when>	Thẻ con của <c:choose> , khi một điều kiện tại when là đúng.
<c:otherwise>	Thẻ con của <c:choose> , khi tất cả các điều kiện <c:when> là sai.
<c:forEach>	Dùng để lặp trên một tập hợp.
<c:forEachTokens>	dùng để lặp trên chuỗi (tokens) được phân cách bởi một dấu phân cách.
<c:param>	Được sử dụng với <c:import> để truyền các tham số.
<c:url>	Để tạo một URL với các tham số tùy chọn (optional query string parameters).



<c:forEach>

```
<%@ taglib uri = "http://java.sun.com/jsp/jstl/core" prefix = "c" %>

<html>
  <head>
    <title><c:forEach> Tag Example</title>
  </head>

  <body>
    <c:forEach var = "i" begin = "1" end = "5">
      Item <c:out value = "${i}"/><p>
    </c:forEach>
  </body>
</html>
```

The above code will generate the following result –

```
Item 1
Item 2
Item 3
Item 4
Item 5
```

<c:forTokens>

Example for <c:forTokens>

```
<%@ taglib uri = "http://java.sun.com/jsp/jstl/core" prefix = "c" %>

<html>
  <head>
    <title><c:forTokens> Tag Example</title>
  </head>

  <body>
    <c:forTokens items = "Zara,nuha,roshy" delims = "," var = "name">
      <c:out value = "${name}"/><p>
    </c:forTokens>
  </body>
</html>
```

The above code will generate the following result –

```
Zara
nuha
roshy
```



c:if

<c:if> được sử dụng để kiểm tra một điều kiện nào đó là đúng hay sai. Chẳng hạn ví dụ dưới đây kiểm tra xem một tập hợp có phần tử hay không (Kiểm tra xem phòng ban hiện tại có nhân viên hay không).

```
<c:if test="${not empty phong.employees }">  
<h3>${phong.deptName}</h3>  
<ul>  
<c:forEach items="${phong.employees }" var="nhanvien">  
<li>  
${nhanvien.empName}-${nhanvien.empNo}  
</li>  
</c:forEach>  
</ul>  
</c:if>
```



c:if

```
<c:if test = "${salary > 2000}">
```

```
<p>My salary is: <c:out value = "${salary}"/><p>
```

```
</c:if>
```

c:choose - c:when - c:otherwise



<c:out>

Thẻ <c:out> hiển thị các kết quả của một biểu thức, tương tự như cách làm việc của <% =%>, khác biệt là <c:out> có thể sử dụng "." để truy cập vào các thuộc tính của đối tượng.

Ví dụ, để truy cập dept.deptName chỉ cần sử dụng thẻ là <c:out value = "\${dept.deptName}" />.

<c:out> có thể tự động escape nội dung XML.



JSTL SQL Tags

- ❑ Thư viện thẻ SQL cho phép thẻ tương tác với RDBMS (Cơ sở dữ liệu quan hệ) chẳng hạn như Microsoft SQL Server, mySQL hoặc Oracle. Cú pháp được sử dụng để bao gồm thư viện thẻ JSTL SQL



JSTL Function Tags

- ❑ JSTL bao gồm các hàm xử lý xâu cơ bản.
- ❑ Sau đây là cú pháp để bao nhóm JSTL Functions.

Hàm	Miêu tả
Hàm fn:contains() trong JSTL	Kiểm tra xâu con
Hàm fn:containsIgnoreCase() trong JSTL	Kiểm tra xâu con, không phân biệt chữ hoa và chữ thường
Hàm fn:endsWith() trong JSTL	Kiểm tra nếu một xâu có kết thúc bởi 1 xâu con
Hàm fn:escapeXml() trong JSTL	Các ký tự được phiên dịch như XML markup
Hàm fn:indexOf() trong JSTL	Trả về index bên trong một xâu về sự xuất hiện đầu tiên của một xâu con



JSP Standard Actions

- ❑ Standard Actions là thẻ (tag) được xây dựng sẵn trong JSP, mỗi thẻ (Standard Actions) được sử dụng để kiểm soát luồng giữa các trang và sử dụng **Java Bean**
- ❑ Nó không cần phải được khai báo với một chỉ thị "taglib". Tất cả các standard action trong JSP có một tiền tố mặc định "jsp".



JSP Standard Actions

Cú pháp	Mục đích sử dụng
jsp:include	Dùng để nhúng một file vào trang JSP tại thời điểm Request. (Chú ý: khác với Directive include là nhúng file vào JSP tại thời điểm biên dịch JSP ra Servlet).
jsp:useBean	Tìm hoặc khởi tạo một đối tượng JavaBean.
jsp:setProperty	Set thuộc tính cho JavaBean
jsp:getProperty	Lấy giá trị của một thuộc tính của JavaBean (Output)
jsp:forward	Chuyển hướng request tới một trang khác.
jsp:element	Định nghĩa các thành phần XML động. (XML elements)
jsp:attribute	
jsp:body	
jsp:text	



JSP Action `jsp:include`

`<jsp:include ..>` cho phép nhúng một trang vào JSP tại thời điểm request. Khác với sử dụng Directive `<%@ include %>` nhúng một file vào JSP tại thời điểm biên dịch JSP thành Servlet.

VD:

JSP Action jsp:useBean

- ❑ JavaBean?
- ❑ JavaBean chính là các lớp thuần Java được sử dụng để thực hiện các thực thể trong chương trình Java
- ❑ JavaBean cần phải tuân thủ theo các quy ước sau:
 - Phải có một constructor không đối số(toán tử tạo lập mặc định)
 - Nó phải được implements từ Serializable.
 - Các thuộc tính là private
 - Mỗi thuộc tính tương ứng với các lớp. Cung cấp phương thức setter/getter để truy cập và thao tác với giá trị các thuộc tính. Trong đó:



JSP Action `jsp:useBean`

- ❑ Nó là một thành phần phần mềm có thể tái sử dụng. Một bean đóng gói nhiều đối tượng vào một đối tượng để chúng ta có thể truy cập đối tượng này từ nhiều nơi. Hơn nữa, nó cung cấp cơ chế giúp bảo trì dễ dàng.



JSP Action `jsp:element`-`jsp:attribute`-`jsp:body`

- ❑ Trong một số tình huống muốn trả dữ liệu xml về phía người dùng thay vì dữ liệu HTML. Việc sử dụng kết hợp `<jsp:element>`, `<jsp:attribute>`, `<jsp:body>` làm cho mã (code) dễ nhìn hơn
- ❑ JSP vốn là một tài liệu với các thẻ kiểu HTML, như vậy tốt nhất hãy sử dụng các thẻ HTML hoặc các thẻ JSP mở rộng thay vì chèn code Java trên JSP. Điều này giúp code trong sáng và dễ nhìn hơn.



Scope trong JSP và Servlet

- ❑ Phạm vi(**Scope**) quyết định khả năng truy cập của một đối tượng hoặc biến. Ví dụ, một số biến có thể truy cập được trong vòng lặp for, khối if-else hoặc trong phương thức hoặc lớp hoặc gói cụ thể.
- ❑ Trong Servlet, scope là định nghĩa về thời gian mà dữ liệu của bạn có tồn tại cho người dùng trong ứng dụng web. Có 4 scope trong servlet là: **page scope**, **request scope**, **session scope** và **application scope**.



Scope trong Servlet

Page Scope

Scope này chỉ có sẵn bên trong page

Request Scope

Data có trong scope này có thể được sử dụng và truy cập cho tới khi response được tạo ra

Session Scope

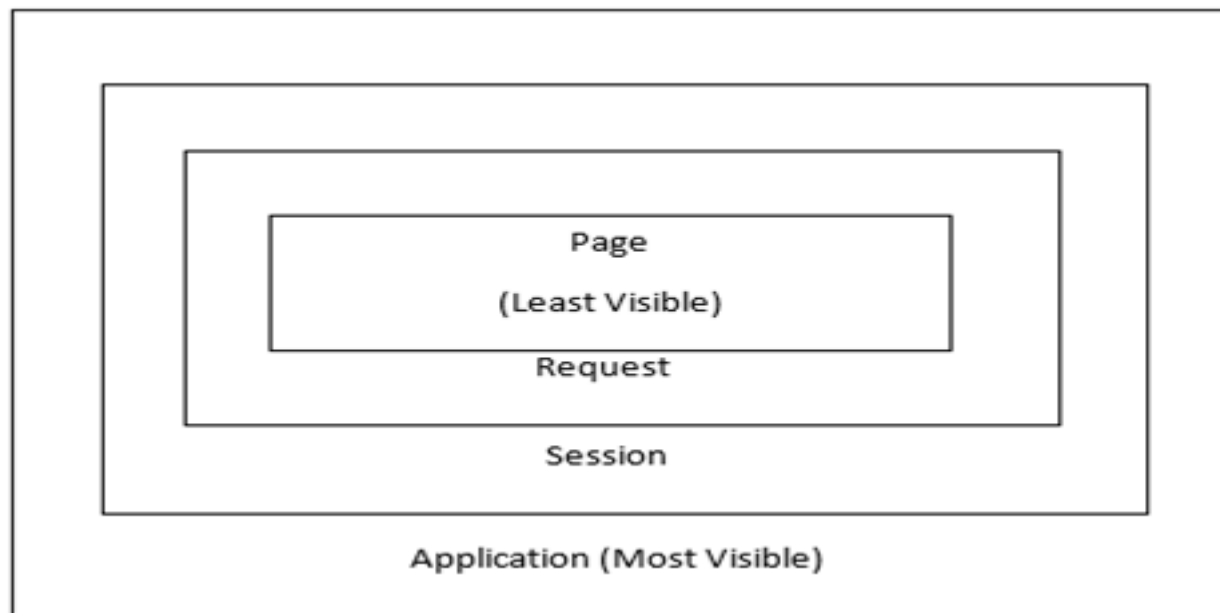
Trong scope này, data tồn tại cho tới khi session bị hủy.

Application Scope

Trong scope này data tồn tại và có thể truy cập từ các session, request khác nhau và chỉ bị hủy khi ứng dụng web bị shutdown



Scope trong Servlet





JSP page scope

- ❑ Page scope giới hạn bởi phạm vi và thời gian tồn tại của các thuộc tính trong cùng 1 page nơi mà nó được tạo.
- ❑ Nó sẵn dùng trong một JSP page giống như một object ẩn.



Request scope

- ❑ Request scope bắt đầu ngay khi một HTTP request được gửi tới server và kết thúc khi server trả về một HTTP response
- ❑ Các tham số/thuộc tính trong một Request scope có thể được truy cập từ các servlet hoặc jsp cùng phục vụ trong 1 request. Ví dụ bạn gọi 1 servlet/jsp sau đó các servlet/jsp này lại gọi các servlet/jsp khác rồi mới trả về response.
- ❑ Request object có sẵn trong JSP page như là 1 object ẩn . Bạn có thể set value cho 1 thuộc tính trong request object từ servlet và lấy nó ra ở JSP (Phải trong cùng 1 request).



Session scope

- ❑ Một Session Scope bắt đầu khi một client thành lập kết nối với ứng dụng web cho tới khi hết thời gian timeout hoặc browser bị đóng.
- ❑ Các giá trị bên trong session scope có thể được truy cập từ nhiều request khác nhau của cùng 1 client
- ❑ Một tính năng đáng chú ý của trình duyệt web (browser) là session được chia sẻ giữa các tab khác nhau. Bạn có thể gửi request từ 1 tab khác mà không cần login lại. Ví dụ bạn login gmail ở 1 tab, sau đó bạn sang tab khác vẫn có thể tải mail, gửi mail mà không cần phải login lại. Do đó, hãy luôn mở một cửa sổ trình duyệt mới khi bạn muốn thực hiện các giao dịch bảo mật.
- ❑ Session object sẵn dùng trong JSP pages giống như một object ẩn
- ❑ Trong Servlet, bạn có thể lấy đối tượng object bằng cách gọi `request.getSession()`.



Application / context scope

- ❑ Application scope hay context scope bắt đầu từ khi ứng dụng web được chạy cho tới khi shutdown. Các tham số, giá trị (Parameters/attributes) bên trong application scope sẵn dùng với tất cả các request và session.
- ❑ **Application** scope được định nghĩa bởi javax.servlet.**ServletContext** interface.
- ❑ Application object sẵn dùng trong 1 JSP page giống như một object ẩn được gọi bởi application.
- ❑ Trong một servlet, bạn có thể lấy các đối tượng application bằng cách gọi `getServletContext()` bên trong servlets code một cách trực tiếp hoặc gián tiếp qua `getServletConfig().getServletContext()`.
- ❑ Web container cung cấp một đối tượng `ServletContext` cho mỗi ứng dụng web.



LẬP TRÌNH JSP

☐ Bài tập thực hành

Bài 1: Hãy tạo một Servlet để hiển thị thông tin Welcome với URL pattern được cấu hình trong web.xml. Servlet này cần đọc một tham số khởi tạo từ web.xml và hiển thị thông tin đó trong trình duyệt.

Yêu cầu:

1. Tạo một dự án Java Dynamic Web Project trên Eclipse.
2. Cấu hình web.xml để ánh xạ Servlet với URL pattern /welcome.
3. Thêm tham số khởi tạo vào web.xml, ví dụ: welcomeMessage = "Welcome to Servlet Basics!".
4. Sử dụng phương thức doGet() để đọc tham số khởi tạo và hiển thị thông tin trong trình duyệt.
5. Triển khai và chạy ứng dụng trên máy chủ Tomcat.



LẬP TRÌNH JSP

❑ Bài tập thực hành

Bài 1: Tạo một Servlet sử dụng annotation thay cho cấu hình web.xml. Servlet này sẽ xử lý yêu cầu POST từ một form HTML, lấy thông tin nhập vào từ người dùng, và phản hồi thông tin đó.

Yêu cầu:

1. Tạo một Servlet sử dụng annotation với URL pattern /submit.
2. Sử dụng phương thức doPost() để nhận dữ liệu từ form HTML.
3. Tạo một form HTML đơn giản để gửi dữ liệu (tên và email) đến Servlet.
4. Servlet trả về phản hồi hiển thị tên và email của người dùng.
5. Triển khai và kiểm tra trên máy chủ Tomcat.



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM